

阅读文献笔记

阅读范围：SGLang / RadixAttention、vLLM / PagedAttention、Mooncake FAST'25 Trace。

一、SGLang 与 RadixAttention

1. 为什么在线推理要管理 KV Cache？

LLM 推理可以粗略分成 Prefill 和 Decode。Prefill 一次处理输入 prompt，Decode 逐 token 生成输出。为了避免每生成一个 token 都重新计算前面的全部上下文，模型会把之前 token 对应的 Key、Value 保存到 KV Cache 中。

2. RadixAttention 的核心不是“再造一种注意力公式”，而是复用已经算过的 KV。

不同请求经常共享前缀，例如相同 system prompt、多轮对话历史、few-shot 示例等。如果每次都重新 Prefill，相同前缀会被重复计算。SGLang 用 radix tree 组织 prompt 与对应的 KV Cache，新请求进来时匹配最长公共前缀，命中的部分直接复用，只计算新增的后缀。

3. 我对 RadixCache 的理解：

它更像“KV Cache 的索引和复用管理层”。缓存空间不足时还需要淘汰旧缓存，因此问题不只是“存下来”，还包括如何查找、共享和淘汰。

二、vLLM 与 PagedAttention

1. PagedAttention 首先解决的是 KV Cache 的显存管理问题。

不同请求长度不同，而且输出长度在真正生成前往往不能准确知道。如果要求每个请求的 KV Cache 都占一大片连续显存，就容易出现预留浪费和碎片。

2. 它借鉴了操作系统分页的思路。

一个请求的 KV Cache 被拆成固定大小的 block，逻辑上属于同一条序列的 block，物理上不要求连续。系统通过映射关系找到对应 block，因此可以按需分配，减少显存碎片，提高可以同时服务的请求数。

3. 与 SGLang 的关系：

PagedAttention 更偏向“KV Cache 怎么存、怎么分配显存”；RadixAttention 更偏向“哪些 KV 已经算过、不同请求之间怎么复用”。两者关注点不同，但并不冲突：paged KV 可以作为底层存储方式，RadixAttention 在更上层管理共享前缀与复用。

三、Mooncake 与 FAST'25 Trace

Mooncake 的思路是把 KV Cache 放到系统设计的中心。它把 Prefill 和 Decode 进行解耦，同时利用 GPU 之外的 CPU、DRAM、SSD、网络资源构建更大的分布式 KV Cache，目标是在长上下文场景下减少重复计算，并在满足延迟目标的同时提高服务吞吐。

Trace 中我实际用到的字段：

timestamp: 相对到达时间；input_length: 输入 token 数；output_length: 输出 token 数；hash_ids: 经过重映射的前缀 block 哈希，可用于表示可复用的前缀块。官方说明中 block 大小为 512 token，相同 hash_id 代表这些前缀块可以共享 KV Cache。

四、几点理解和疑问

1. 长输入和长输出带来的瓶颈不同。Prefill 更偏计算密集，Decode 更依赖 KV Cache 的持续读取和增长。所以不能只看 input_length 判断总延迟，output_length 也会明显影响 latency。
2. KV Cache 优化其实有两层。一层是显存怎么高效分配，例如 PagedAttention；另一层是缓存内容怎么复用，例如 RadixAttention。把这两层区分开以后，很多概念就不容易混。
3. trace 不等于原始用户文本。Mooncake 公布的是匿名化后的长度、时间和前缀哈希信息，因此实验中需要自己构造 synthetic prompt。这样既能复现请求形态，又不会暴露真实用户内容。
4. 我现在还没有深入到内核级实现。例如 PagedAttention 的 CUDA kernel、RadixAttention 在不同 page size 下的数据结构和淘汰策略，只做到了理解系统层面的作用。

五、参考资料

1. SGLang: Efficient Execution of Structured Language Model Programs, NeurIPS 2024.
2. SGLang Documentation - serving / benchmarking / cache related documentation.
3. Efficient Memory Management for Large Language Model Serving with PagedAttention, SOSP 2023 (vLLM).
4. vLLM Documentation - PagedAttention and paged KV cache.
5. Mooncake: Trading More Storage for Less Computation - A KVCache-centric Architecture for Serving LLM Chatbot, FAST '25.
6. Mooncake FAST'25 Trace Release README.